

# 变物性与收缩条件下药材烘干的传热传质模型

## 摘 要

药材烘干时，热量由表面传入，水分由内部向外迁移，二者的速率并不相同；同时，物性变化和几何收缩会影响干燥终点。为描述这一过程，本文建立圆柱形药材的一维径向传热传质模型，采用环形有限体积法离散空间，并用隐式时间积分和Richardson外推求解。

关于问题一，在固定半径和常热物性条件下，分别求解温度场和含水率场。计算得到1800 s时中心、表面温度分别为33.5753 °C和36.7856 °C，对应含水率分别为2.5500 kg/kg和1.5102 kg/kg。由此可见，预热阶段中心水分变化很小，失水主要集中在表层。

关于问题二，将题设中的物性参数表示为温度和含水率的函数，建立温湿耦合模型。3 h时中心、表面温度分别为49.8495 °C和49.9664 °C，含水率分别为1.7662 kg/kg和1.0081 kg/kg。可以看出，此时温度已经接近均匀，但水分仍有明显的径向梯度，升温完成后还需要继续干燥。

关于问题三，以附件1最后一小时的环境均值延拓长期边界，并把全域最大含水率低于0.15 kg/kg作为干燥完成的判据。固定半径模型的临界时长为57.4740 h，首个严格达标整分钟为57.4833 h。在不同长期环境设定下，临界时长约为57.24—57.83 h，说明结果对环境延拓方式较为敏感。

关于问题四，根据均匀收缩条件下的干物质和水分守恒建立移动边界模型，并用归一化径向坐标把变化区域映射到固定区间。计算得到临界时长为51.0920 h，首个严格达标整分钟为51.1000 h。若保持本问物性而固定初始半径，对照时长为129.8481 h，说明收缩会明显加快本模型中的干燥过程。

网格加密、通量平衡和独立离散计算均支持上述结果。两类终点的相邻外推值相差约1 s。需要指出的是，所得时长依赖一维近似、有效传质边界以及经验密度与收缩关系等假设，应理解为相应模型条件下的预测。

关键词：药材烘干；传热传质；有限体积法；移动边界；干燥终点

# 1 问题重述与分析

## 1.1 任务及数据

题目研究一根长 25 cm、初始半径为 2 cm 的圆柱形药材。烘干开始时，药材温度为 28 °C，干基含水率为 2.55 kg/kg；附件 1 给出了烘房初期的温度和含湿量，附件 2 给出了药材半径随时间的变化。题目既要求考察整体干燥趋势，又要求给出指定位置的温度和含水率，因此需要建立具有空间分辨能力的模型 [1]。

四个问题分别考察预热过程、变物性干燥、干燥终点和几何收缩。问题一计算固定半径下 30 min 内的温度和含水率；问题二从题设初值出发，采用附录 3 物性计算全程温湿分布，并列前三小时结果；问题三沿用问题二模型，确定各处含水率均低于 0.15 kg/kg 的时间；问题四采用附录 4 物性和实测半径，重新计算干燥终点。问题二从初始时刻独立计算，问题三是在此基础上的长期延伸。

## 1.2 模型选择

本题要求给出药材内部的局部分布和全域达标时间，仅用平均含水率经验曲线无法完成上述任务。为此，本文采用 Fourier 导热方程和 Fick 有效扩散方程描述内部输运，再用表面对流边界连接药材与烘房环境。圆柱物料的热质耦合模型 [2] 以及非等温移动边界模型 [3] 为模型选择提供了参考，具体物性和观测数据均采用题目给定值。

药材长径比为 6.25，本文把径向结果理解为圆柱中截面的分布，并采用一维轴对称近似。建模时先考虑常物性，再引入变物性，最后加入移动边界，逐步使用题目提供的信息。问题四直接把半径观测作为输入，不再拟合额外的收缩方程；这与根据局部收缩速度预测几何的文献模型 [3] 不同。

# 2 模型假设与符号

## 2.1 基本假设

(1) 药材为径向各向同性的轴对称连续介质，忽略周向差异和中段轴向梯度，以一维径向场近似中截面分布。

(2) 第一至第三问半径不变；第四问长度不变、截面均匀径向收缩，整体半径历史适用于所研究的中截面。

(3) 初始温度和干基含水率均匀。热物性与扩散率按各问指定附录取局部状态值，不另行拟合参数。

(4) 附件 1、附件 2 均作分段线性插值。4 h 后的烘房环境以最后一小时均值为基准平台，末观测之后用 60 s 过渡到平台；这是长期环境假设。

(5) 第二至第四问未另给表面系数，故沿用第一问的传热系数  $25 \text{ W}/(\text{m}^2\cdot\text{K})$  和传质系数  $8\times 10^{-7} \text{ m/s}$ 。气相含湿量与固相干基含水率的质量基准不同，在缺少吸附等温线时，将二者之差解释为有效传质推动力。

(6) 忽略蒸发潜热、热湿交叉迁移和力学应力，以经验密度和比热的乘积表示有效体积热容。模型按有效介质解释，其质量与热力学闭合限制见第 8.5 节。

(7) 第四问干物质无损失、初始干物质体积密度均匀；在均匀收缩下，该干密度只随时间变化。此处的守恒干密度与附录经验密度分开定义，二者的相容性在第 8 节检验。

## 2.2 主要符号

| 符号                   | 含义                 | 单位                                  |
|----------------------|--------------------|-------------------------------------|
| $t$                  | 自烘干起点计的时间          | s                                   |
| $r, R(t), R_0$       | 物理半径、当前表面半径、初始表面半径 | m                                   |
| $\xi = r/R(t)$       | 随材料运动的归一化径向坐标      | 1                                   |
| $T, T_K$             | 摄氏温度、开尔文温度         | °C、K                                |
| $C$                  | 固相干基含水率            | kg/kg                               |
| $T_\infty, C_\infty$ | 烘房温度、环境含湿量         | °C、kg/kg                            |
| $\rho, c_p, k$       | 经验密度、比热容、导热系数      | kg/m <sup>3</sup> 、J/(kg·K)、W/(m·K) |
| $D$                  | 有效水分扩散率            | m <sup>2</sup> /s                   |
| $h_T, h_m$           | 对流传热、有效传质系数        | W/(m <sup>2</sup> ·K)、m/s           |
| $\rho_d, v_r$        | 当前干物质体积密度、径向骨架速度   | kg/m <sup>3</sup> 、m/s              |
| $t_*, t_{60}$        | 临界时刻、其后首个严格达标整分钟   | s                                   |

所有计算距离使用 m，表格按题意显示 cm；内部温度保存为 °C，只在扩散率的 Arrhenius 项中换算为 K。后文的“水分浓度”均指题目定义的干基含水率。

## 3 数据准备与统一数值方法

### 3.1 数据核验及环境延拓

附件 1 共有 241 组记录，时间范围为 0—14400 s，采样间隔为 60 s；附件 2 共有 145 组记录，时间范围为 0—259200 s，采样间隔为 1800 s。经核对，提取后的两份 CSV 与原始工作簿逐值相同。药材半径由 2.000 cm 逐渐减小到 1.198 cm，计算时直接使用原始观测值，不另用拟合曲线代替。

对相邻观测点，环境或半径变量  $f$  采用

$$f(t) = f_i + \frac{t-t_i}{t_{i+1}-t_i} (f_{i+1} - f_i), t_i \leq t \leq t_{i+1} \quad (1)$$

附件 1 只记录了前 4 h 的烘房环境，而问题三、四需要计算更长时间。最后一小时共有 61 个观测点，温度和含湿量的均值分别为 49.998934 °C 和 0.04998754 kg/kg，标准差分别为 0.154353 °C 和 0.00015013 kg/kg；相应线性趋势约为 0.002824 °C/h 和 -0.00001288 kg/(kg·h)。这一时段波动较小，因此取其均值作为后续环境平台，并在第 8.3 节比较其他延拓方式。

### 3.2 环形有限体积离散

为了计算温度和含水率随半径的变化，在各径向节点周围划分环形控制体。省略各单元共有的轴向长度后，控制体体积为  $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ ，外侧面积为  $A_{i+1/2} = 2\pi r_{i+1/2}$ 。对一般传输变量  $u$ ，记蓄积系数为  $s$ 、输运系数为  $\Gamma$ ，可得半离散方程

$$s_i V_i \frac{du_i}{dt} = A_{i+1/2} \Gamma_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r} - A_{i-1/2} \Gamma_{i-1/2} \frac{u_i - u_{i-1}}{\Delta r} \quad (2)$$

对热传导取  $u = T, s = \rho c_p, \Gamma = k$ ；对有效水分扩散取  $u = C, s = 1, \Gamma = D$ 。界面系数采用调和平均：

$$\Gamma_{i+1/2} = \frac{2\Gamma_i \Gamma_{i+1}}{\Gamma_i + \Gamma_{i+1}} \quad (3)$$

圆心控制体的内侧面积为零，可以自然处理  $r = 0$  处的对称条件；最外层控制体则用题设的 Robin 通量代替外侧通量。相邻控制体交界面上的通量大小相等、方向相反，因而对所有单元求和后只剩表面通量，这也为后续的平衡检验提供了依据。

### 3.3 时间积分、正性与外推

空间离散后，偏微分方程转化为刚性常微分方程组，本文采用后向差分公式（BDF）进行隐式积分。为避免计算中出现负含水率，令  $z = \ln C$ ，按  $z_t = C_t / C$  更新，再由  $C = \exp z$  还原含水率。积分器自动调整步长，并在题目要求的时刻输出结果。

在共同的物理输出节点上，对间隔数为  $N$ 、 $2N$  的两级解作二阶 Richardson 外推：

$$u_{\text{ext}} = \frac{4u_{2N} - u_N}{3} \quad (4)$$

问题一采用 5120/10240 两级网格，问题二、三采用 2560/5120 两级网格，问题四采用 640/1280 两级网格。计算结果经式（4）外推后，再按题意保留四位小数。问题二每 600 s 分段积分，段与段之间传递未舍入的完整节点值，从而避免舍入误差累积。

## 4 问题一：固定半径预热模型

### 4.1 方程、初值与边界

在  $0 < r < R_0 = 0.02$  m 内，采用

$$\begin{aligned}\rho c_p \frac{\partial T}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left( r k \frac{\partial T}{\partial r} \right) \\ \frac{\partial C}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left( r D(C) \frac{\partial C}{\partial r} \right)\end{aligned}\quad (5)$$

附录 2 给定  $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg}\cdot\text{K)}$ 、 $k = 0.36 \text{ W/(m}\cdot\text{K)}$ ，水分扩散率为

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s} \quad (6)$$

初始条件为  $T(r, 0) = 28 \text{ }^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ ；圆心满足  $T_r(0, t) = C_r(0, t) = 0$ 。径向向外取正，表面边界为

$$\begin{aligned}-kT_r(R_0, t) &= h_T[T_s - T_\infty(t)] \\ -DC_r(R_0, t) &= h_m[C_s - C_\infty(t)]\end{aligned}\quad (7)$$

由式（7）可知，当烘房温度高于药材表面温度时，热量由环境进入药材；当表面含水率高于环境有效含湿量时，水分由药材向外迁移。问题一中的热物性为常数，扩散率也不随温度变化，因此温度方程和水分方程可以分别求解。

#### 4.2 指定位置的计算结果

将上述方程代入附件 1 的环境数据进行计算，得到表 1、表 2。表中列出题目指定时刻和位置的结果，完整的 1—1800 s 温度场和含水率场保存在支撑材料 result1.xlsx 中，径向间隔为 0.1 cm。所有结果均按题意保留四位小数。

表 1 代表位置温度  $T/ \text{ }^\circ\text{C}$

| 时间/s | 0 cm    | 0.5 cm  | 1.0 cm  | 1.5 cm  | 2.0 cm  |
|------|---------|---------|---------|---------|---------|
| 100  | 28.0001 | 28.0003 | 28.0040 | 28.0327 | 28.1801 |
| 300  | 28.0408 | 28.0635 | 28.1514 | 28.3681 | 28.8489 |
| 600  | 28.4533 | 28.5360 | 28.8039 | 29.3159 | 30.1652 |
| 900  | 29.3243 | 29.4583 | 29.8755 | 30.6161 | 31.7304 |
| 1200 | 30.5427 | 30.7098 | 31.2223 | 32.1126 | 33.4276 |
| 1500 | 31.9957 | 32.1867 | 32.7660 | 33.7463 | 35.1203 |
| 1800 | 33.5753 | 33.7720 | 34.3642 | 35.3621 | 36.7856 |

表 2 代表位置水分浓度  $C/(kg/kg)$

| 时间/s | 0 cm   | 0.5 cm | 1.0 cm | 1.5 cm | 2.0 cm |
|------|--------|--------|--------|--------|--------|
| 100  | 2.5500 | 2.5500 | 2.5500 | 2.5500 | 2.2470 |
| 300  | 2.5500 | 2.5500 | 2.5500 | 2.5492 | 2.0508 |
| 600  | 2.5500 | 2.5500 | 2.5500 | 2.5353 | 1.8770 |
| 900  | 2.5500 | 2.5500 | 2.5497 | 2.5045 | 1.7547 |
| 1200 | 2.5500 | 2.5500 | 2.5482 | 2.4646 | 1.6586 |
| 1500 | 2.5500 | 2.5499 | 2.5445 | 2.4206 | 1.5787 |
| 1800 | 2.5500 | 2.5497 | 2.5383 | 2.3755 | 1.5102 |

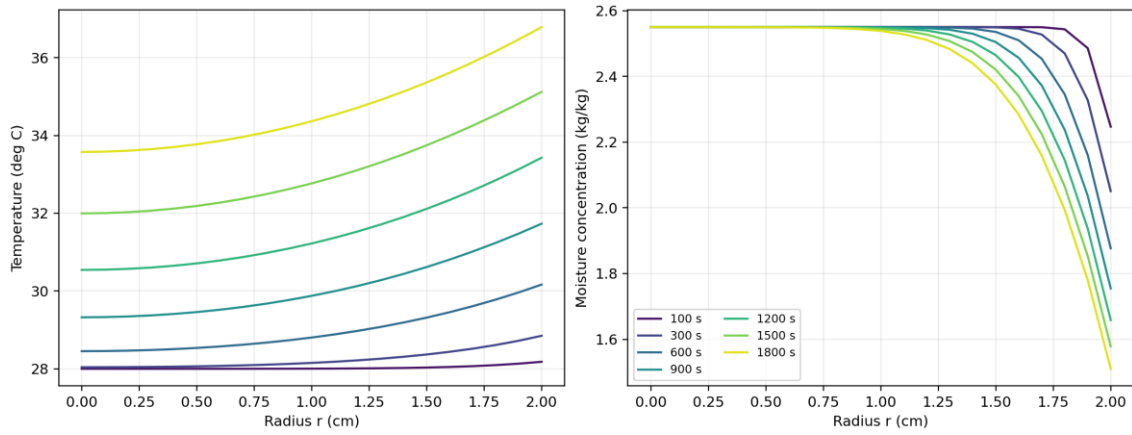


图 1 预热阶段的径向温度与含水率分布；不同曲线对应图例给出的时刻。

由图 1 可以看到，随着烘干时间增加，热量逐渐由表面传向中心，而水分损失主要发生在表层。1800 s 时，表面与中心的温差为  $3.2103\text{ }^{\circ}\text{C}$ ；表面含水率已经降至  $1.5102\text{ kg/kg}$ ，中心含水率仍接近初值。说明在预热阶段，温度传播快于内部水分迁移，药材表层先出现明显失水。

## 5 问题二：变物性温湿耦合模型

### 5.1 经验物性与耦合关系

从烘干初始时刻统一采用附录 3 物性 [1]:

$$\rho = 650 + 128C, c_p = 1450 + 2736 \frac{C}{C+1}, k = 0.21 + 0.38 \frac{C}{C+1} \quad (8)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right), T_K = T + 273.15 \quad (9)$$

把式 (8) — (9) 代入式 (5)，并结合题设初值和式 (7) 的边界条件，即可得到问题二的温湿耦合模型。含水率变化会引起密度、比热和导热系数变化，温度又通过扩散率影响水分迁移，因此两个方程需要联立求解。计算 Arrhenius 项时，将药材温度由摄氏度换算为开尔文温度。

这里将水分迁移看作固定骨架中的有效扩散过程，并用经验量  $\rho(C)c_p(C)$  表示体积热容。若进一步把  $\rho(C)/(1 + C)$  解释为严格守恒的干物质密度，会与后续收缩关系产生矛盾，因此本文只把它作为有效物性使用，具体相容性见第 8.5 节。

5.2 前三小时结果与全程输出

表 3 3 小时内药材温度 (°C)

| 时间/h | 0 cm    | 0.5 cm  | 1.0 cm  | 1.5 cm  | 2.0 cm  |
|------|---------|---------|---------|---------|---------|
| 0.5  | 32.1892 | 32.3821 | 32.9659 | 33.9608 | 35.4130 |
| 1.0  | 40.3816 | 40.5536 | 41.0605 | 41.8782 | 42.9976 |
| 1.5  | 45.8468 | 45.9348 | 46.1930 | 46.6051 | 47.1400 |
| 2.0  | 48.4502 | 48.4881 | 48.5984 | 48.7736 | 49.0033 |
| 2.5  | 49.4670 | 49.4792 | 49.5136 | 49.5653 | 49.6609 |
| 3.0  | 49.8495 | 49.8553 | 49.8746 | 49.9101 | 49.9664 |

表 4 3 小时内药材水分浓度 (kg/kg)

| 时间/h | 0 cm   | 0.5 cm | 1.0 cm | 1.5 cm | 2.0 cm |
|------|--------|--------|--------|--------|--------|
| 0.5  | 2.5499 | 2.5489 | 2.5256 | 2.3257 | 1.6486 |
| 1.0  | 2.5257 | 2.4948 | 2.3578 | 2.0230 | 1.4711 |
| 1.5  | 2.3861 | 2.3256 | 2.1344 | 1.8020 | 1.3476 |
| 2.0  | 2.1709 | 2.1084 | 1.9236 | 1.6259 | 1.2311 |
| 2.5  | 1.9566 | 1.9006 | 1.7360 | 1.4720 | 1.1166 |
| 3.0  | 1.7662 | 1.7165 | 1.5702 | 1.3333 | 1.0081 |

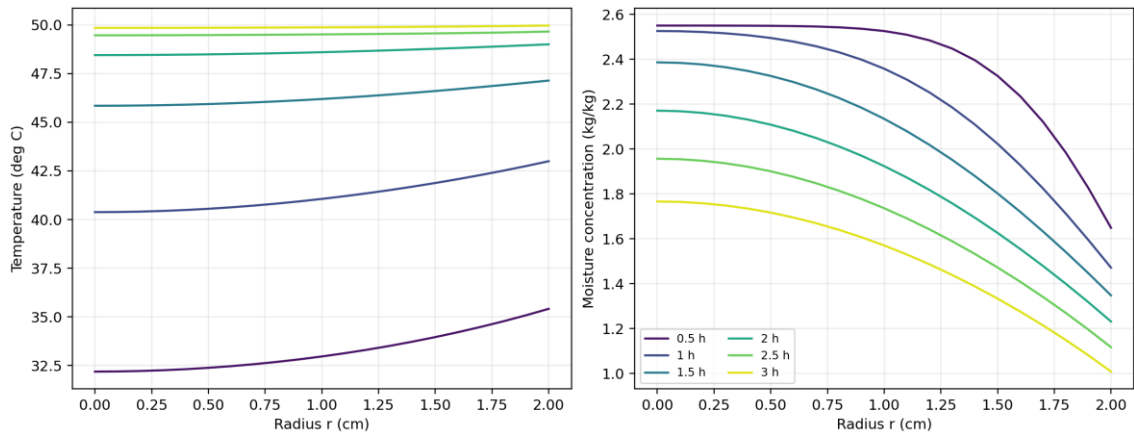


图 2 变物性模型前三小时的径向温湿分布。

从表 3、表 4 和图 2 可以看出，3 h 时中心与表面的温差只有 0.1169 °C，药材内部温度已经接近均匀；但中心与表面的含水率仍相差 0.7581 kg/kg，且各处都没有达到 0.15 kg/kg。随着表层含水率降低，扩散率继续减小，内部水分向外迁移还需要较长时间。因此，表面温度趋于稳定并不意味着烘干已经完成。

表 3、表 4 给出了前三小时的计算结果。全程温湿场按 1 s 时间间隔、0.1 cm 径向间隔输出，并一直计算到问题三的首个严格达标整分钟。长期环境按第 3.1 节处理，具体输出和跨问题核对方式见附录 A.2。

## 6 问题三：固定半径的干燥终点

### 6.1 全域阈值与严格不等式

沿用第二问方程，定义

$$g(t) = \max_{0 \leq r \leq R_0} C(r, t) - 0.15, t_* = \inf\{t: g(t) < 0\} \quad (10)$$

问题三要求药材各处的含水率都低于 0.15 kg/kg，因此不能只考察表面值或平均值。计算时遍历全部径向节点，并以最大含水率判断是否达标。结果表明，含水率由中心向表面逐渐减小，所以本题的干燥终点由中心位置决定。在临界时刻有  $g(t_*) = 0$ ，严格低于阈值则发生在其后。

为准确确定终点，本文在阈值两侧分别对各径向位置的未舍入含水率进行时间插值，并取最晚达到阈值的位置。若设备只能按整分钟停止，则取临界时刻之后的第一个整分钟，即

$$t_{60} = 60 \left( \left\lfloor \frac{t_*}{60} \right\rfloor + 1 \right) \quad (11)$$

### 6.2 时长与指定结果

第三问临界时刻为 206906.2447 s（57.4740 h），首个严格达标整分钟为 206940 s（57.4833 h）。

表 5 药材烘干过程的水分浓度（kg/kg）

| 时间/h            | 0 cm   | 0.5 cm | 1.0 cm | 1.5 cm | 2.0 cm |
|-----------------|--------|--------|--------|--------|--------|
| 6               | 1.0170 | 0.9881 | 0.9015 | 0.7550 | 0.5328 |
| 12              | 0.4561 | 0.4432 | 0.4031 | 0.3297 | 0.1642 |
| 18              | 0.2988 | 0.2916 | 0.2687 | 0.2253 | 0.0845 |
| 24              | 0.2378 | 0.2327 | 0.2167 | 0.1855 | 0.0665 |
| 30              | 0.2058 | 0.2018 | 0.1892 | 0.1641 | 0.0598 |
| 36              | 0.1859 | 0.1825 | 0.1718 | 0.1504 | 0.0566 |
| 42              | 0.1721 | 0.1691 | 0.1597 | 0.1407 | 0.0548 |
| 48              | 0.1618 | 0.1592 | 0.1507 | 0.1334 | 0.0537 |
| 54              | 0.1539 | 0.1514 | 0.1436 | 0.1277 | 0.0530 |
| 临界终点<br>57.4740 | 0.1500 | 0.1477 | 0.1402 | 0.1249 | 0.0526 |

表 5 最后一行给出了临界时刻的含水率，此时中心显示为 0.1500，恰好位于阈值边界。按式（11）计算，下一整分钟才严格满足题目要求。完整工作簿同时保留了分钟序列、临界行和首个严格达标整分钟行。

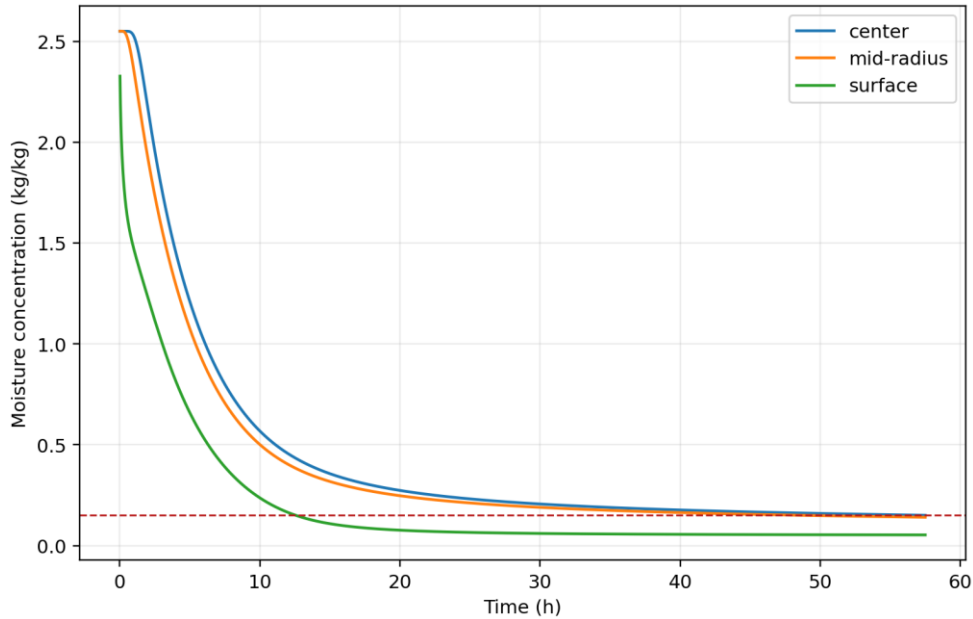


图 3 固定半径下中心、中半径和表面的长期含水率历程；虚线为 0.15 kg/kg 阈值。

由图 3 可以看到，表面含水率下降最快，中心下降最慢；进入干燥后期后，各位置的变化逐渐变缓。因此，只判断表面或平均含水率会提前结束烘干，必须以全域最大值作为终点判据。上述时长是在第 3.1 节长期环境假设下得到的，其敏感性将在第 8.3 节分析。

## 7 问题四：收缩移动边界模型

### 7.1 从干物质与水分守恒出发

问题四与前三问的主要区别是药材半径随时间变化。附件 2 已经给出  $R(t)$ ，因此直接把实测半径引入模型。在均匀径向收缩且长度不变的条件下，材料速度为  $v_r = (R/R)r$ 。设当前干物质体积密度为  $\rho_d$ ，相对骨架的水分通量为  $\mathbf{j}_w$ ，则干物质和水分分别满足

$$\begin{aligned}\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}) &= 0 \\ \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v} + \mathbf{j}_w) &= 0\end{aligned}\quad (12)$$

取有效 Fick 通量  $\mathbf{j}_w = -\rho_d D \nabla C$ ，并从上述两式中消去干物质连续方程，得到

$$\frac{DC}{Dt} = \frac{1}{\rho_d} \nabla \cdot (\rho_d D \nabla C) \quad (13)$$

由于初始干密度均匀，且截面作均匀径向收缩，干物质守恒给出  $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$ 。此时干密度只随时间变化，式（13）中的  $\rho_d$  可以约去，从而得到

$$\frac{\partial C}{\partial t} + \frac{R}{r} \frac{\partial C}{\partial r} = \frac{1}{r} \frac{\partial}{\partial r} \left( r D \frac{\partial C}{\partial r} \right) \quad (14)$$

式 (14) 中的  $C$  是单位干物质质量所含的水分，几何收缩已经通过骨架运动和干物质守恒体现，因此不需要再加入体积压缩源项。

## 7.2 固定域坐标与表面通量

为避免不断改变计算区域，令  $\xi = r/R(t)$ ，把收缩的物理区域映射到固定区间  $0 < \xi < 1$ 。由于固定  $\xi$  的网格速度与骨架速度相同，坐标变换产生的项恰好与式 (14) 中的对流项抵消，于是得到

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} \Big|_{\xi} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi k(C) \frac{\partial T}{\partial \xi} \right) \quad (15)$$

$$\frac{\partial C}{\partial t} \Big|_{\xi} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi D(C, T) \frac{\partial C}{\partial \xi} \right) \quad (16)$$

热方程仍采用有效热容近似，不计变形功和相变热。圆心满足对称条件  $T_{\xi} = C_{\xi} = 0$ ，实际表面的边界条件为

$$\begin{aligned} -\frac{k}{R} T_{\xi}(1, t) &= h_T (T_s - T_{\infty}) \\ -\frac{D}{R} C_{\xi}(1, t) &= h_m (C_s - C_{\infty}) \end{aligned} \quad (17)$$

从式 (15) — (17) 可以看出，内部扩散项随  $R^{-2}$  变化，表面通量随  $R^{-1}$  变化。随着半径减小，内部传递距离缩短，收缩因而会影响干燥速度。

附录 4 的局部物性为

$$\rho = 760 + 90C, c_p = 1850 + 2150 \frac{C}{C+1}, k = 0.12 + 0.20 \frac{C}{C+1} \quad (18)$$

$$D(C, T) = 4.2 \times 10^{-4} \exp \left( -\frac{0.30}{C} \right) \exp \left( -\frac{3850}{T_K} \right) \quad (19)$$

## 7.3 固定物理位置的映射与结果

题目要求给出固定物理位置的含水率，因此在每个时刻用  $\xi = r/R(t)$  把相应位置映射到计算网格。当  $r > R(t)$  时，该位置已经位于药材之外，工作簿中的相应单元格留空；“药材表面”一行始终取  $\xi = 1$ ，随真实边界移动。

第四问临界时刻为 183931.0374 s (51.0920 h)，首个严格达标整分钟为 183960 s (51.1000 h)。

表 6 收缩药材烘干过程的水分浓度 (kg/kg)

| 时间/h                | 0 cm   | 0.5 cm | 1.0 cm | 药材表面   |
|---------------------|--------|--------|--------|--------|
| 6                   | 1.7196 | 1.5377 | 1.0226 | 0.4207 |
| 12                  | 0.7377 | 0.6546 | 0.4083 | 0.1669 |
| 18                  | 0.4088 | 0.3687 | 0.2397 | 0.0892 |
| 24                  | 0.2851 | 0.2611 | 0.1790 | 0.0673 |
| 30                  | 0.2263 | 0.2094 | 0.1493 | 0.0595 |
| 36                  | 0.1928 | 0.1797 | 0.1317 | 0.0560 |
| 42                  | 0.1712 | 0.1604 | 0.1200 | 0.0541 |
| 48                  | 0.1561 | 0.1468 | 0.1116 | 0.0530 |
| 烘干临界时刻<br>51.091955 | 0.1500 | 0.1413 | 0.1081 | 0.0526 |

由附件 2 可知, 6 h 时药材半径已经减小到 1.374 cm, 此后还将继续减小。因此, 在表 6 所列时刻, 距中心 1.5 cm 和 2 cm 的位置均已位于药材之外, 表中只列出 0、0.5、1 cm 和实际表面。完整的 result4.xlsx 仍保留所有间隔为 0.1 cm 的固定位置, 并用空白表示实体外位置。

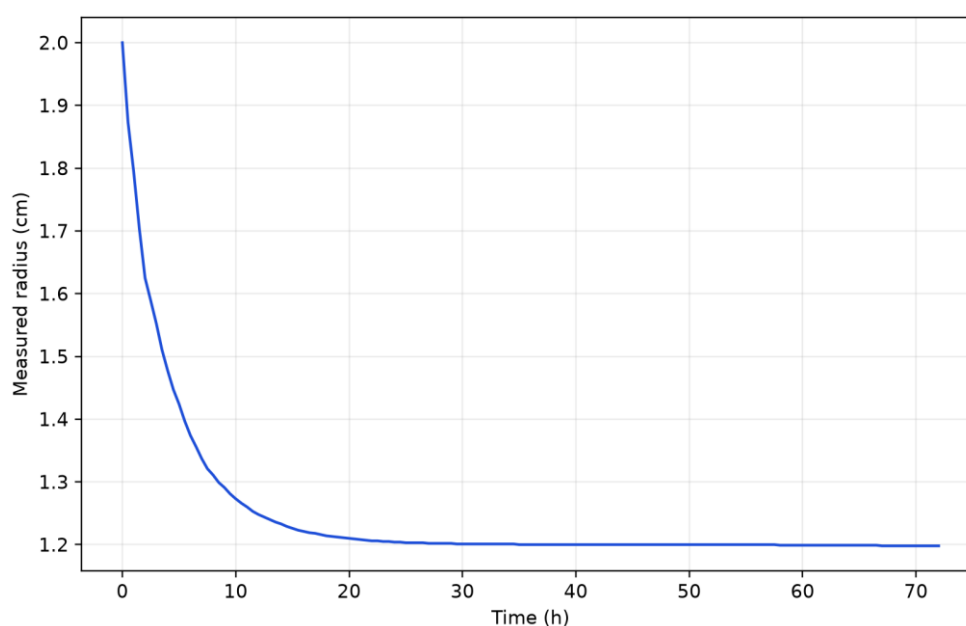


图 4 附件 2 的半径历史及分段线性插值。

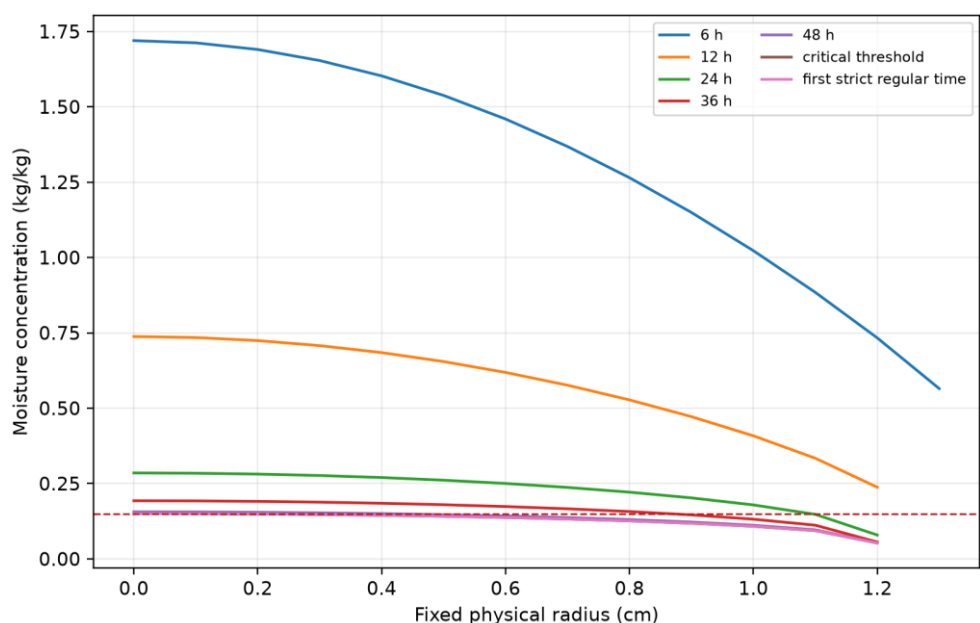


图 5 收缩药材在代表时刻的径向含水率；每条曲线右端是当时实际表面。

问题三和问题四的临界时长相差约 6.38 h，但两问同时改变了物性和几何条件，不能把这一差值全部归因于收缩。为进一步考察几何收缩的作用，第 8.4 节保持附录 4 物性不变，只把半径固定为初始值进行对照。

## 8 模型检验与灵敏度分析

### 8.1 网格与程序一致性

首先对空间网格进行加密计算。第一问两级网格在全部输出节点的最大温度和含水率差分别约为  $1.03 \times 10^{-6} \text{ }^{\circ}\text{C}$  和  $4.57 \times 10^{-6} \text{ kg/kg}$ ；第二问前三小时的最大差分别约为  $8.14 \times 10^{-7} \text{ }^{\circ}\text{C}$  和  $1.49 \times 10^{-5} \text{ kg/kg}$ 。由此可见，网格加密后结果变化很小，本文取式 (4) 的外推值作为最终计算结果。

表 7 两类终点的空间加密结果

| 模型  | 径向区间或外推组合    | 临界时长/h     |
|-----|--------------|------------|
| 第三问 | 1280         | 57.4881201 |
| 第三问 | 2560         | 57.4772860 |
| 第三问 | 5120         | 57.4747882 |
| 第三问 | 2560/5120 外推 | 57.4739569 |
| 第四问 | 640          | 51.0996704 |
| 第四问 | 1280         | 51.0938837 |
| 第四问 | 640/1280 外推  | 51.0919548 |

表 7 中第三、四问相邻外推组合的终点差分别约为 1.02 s 和 0.98 s。第四问再采用单元中心有限体积程序独立计算，1600/3200 单元外推得到 51.0919474 h，表 6 的代表

时刻结果与主程序保留四位小数后的结果一致。以上结果说明数值计算具有较好的一致性；考虑到实际观测收敛阶低于 2，外推差只作为误差量级参考。

此外，程序还检验了平衡场保持、边界通量方向、固定半径退化和阈值插值等情形，共有 95 项自动化测试通过。表 1—6 与数值输出一致，工作簿回读和跨问题比较的详细记录见附录 A.2。

8.2 水分通量平衡

固定半径下，截面平均含水率满足

$$\overline{C}(t) - \overline{C}(0) = -\frac{2h_m}{R_0} \int_0^t [C_s(\tau) - C_\infty(\tau)] d\tau \tag{20}$$

移动域中定义  $\overline{C} = 2 \int_0^1 C \xi d\xi$ ，由式 (16) — (17) 得

$$\frac{d\overline{C}}{dt} = -\frac{2h_m}{R(t)} [C_s(t) - C_\infty(t)] \tag{21}$$

利用前 600 s 的逐秒输出和后续分钟输出，对边界通量进行积分。第三问两级网格的相对最大残差约为  $1.21 \times 10^{-6}$ ，第四问分别约为  $1.61 \times 10^{-6}$  和  $1.69 \times 10^{-6}$ 。残差均较小，说明离散结果与模型自身的水分通量平衡关系相符。

8.3 长期环境与表面系数情景

表 8 第三问长期平台敏感性（同为 640 区间）

| 4 h 后环境设定          | 临界时长/h  |
|--------------------|---------|
| 最后一小时均值            | 57.5413 |
| 最后半小时均值            | 57.5151 |
| 最后观测值保持            | 57.2375 |
| 温度降低、含湿量升高各一个尾段标准差 | 57.8259 |
| 温度升高、含湿量降低各一个尾段标准差 | 57.2587 |

表 8 中各情景采用相同网格，临界时长的跨度约为 0.59 h，明显大于相邻网格外推的差异。可见，长期环境设定对终点的影响大于当前空间离散误差；表中范围是指定情景下的预测变化，不是统计置信区间。

表 9 第四问表面系数敏感性（同为 320 区间）

| 变化系数 | 基准的 0.8 倍/h | 基准/h    | 基准的 1.2 倍/h |
|------|-------------|---------|-------------|
| 传质系数 | 51.8009     | 51.1236 | 50.7400     |
| 传热系数 | 51.1418     | 51.1236 | 51.1117     |

由表 9 可以看出，在给定取值范围内，传质系数变化引起的终点偏移明显大于传热系数变化的影响。这说明干燥后期主要受水分迁移控制。0.8 倍和 1.2 倍只是设定的扰动幅度，实际参数还需要实验标定。

## 8.4 收缩处理与固定几何对照

在保持附件 2 观测点不变的情况下，将分段线性插值改为单调 PCHIP，320 区间计算得到的终点提前约 12.603 s。进一步把半径观测时刻统一提前或推后 15 min，终点相对同组基准分别改变约 -0.135 h 和 +0.137 h。前者考察插值方法，后者考察观测时标扰动，两种检验分别反映不同的不确定性来源。

在附录 4 物性保持不变、而把半径固定为  $R = R_0$  的对照模型中，640/1280 区间外推终点为 129.8481 h，而实测收缩模型约为 51.0920 h。半径减小同时改变式 (16) 的内部扩散尺度和式 (17) 的表面尺度，因此会显著影响水分迁移。该对照只用于分离所选模型中的几何因素，并不等同于真实药材收缩效应的独立实验测量。

## 8.5 密度与收缩的相容性

为了检验密度关系与收缩假设是否相容，先把附录 4 的  $\rho(C)$  解释为当前真实湿基密度。此时有  $\rho_d = \rho(C)/(1 + C)$ ，相应的总干物质质量为

$$M_d(t) = 2\pi LR(t)^2 \int_0^1 \frac{760+90C(\xi,t)}{1+C(\xi,t)} \xi d\xi \quad (22)$$

对非均匀含水率场进行积分，在 1280 区间的临界时刻得到  $M_d(t_*)/M_d(0) \approx 0.890289$ ，相应偏差约为 10.97%。这说明，若把给定密度看作真实湿基密度，则长度不变的实测收缩与干物质守恒在当前计算中不能同时严格成立。这里的偏差只用于检验模型闭合关系，不表示药材实际损失了相同比例的干物质。

本文在热方程中将附录密度用于经验有效体积热容，在水分方程中则按均匀收缩假设确定干密度。这样的处理可以构成当前题设下的有效模型，但密度与几何关系的不相容仍然是模型的结构限制。若要建立严格的多组分守恒模型，还需补充孔隙率、局部变形或长度变化等信息，现有附件尚不能确定这些关系。

# 9 模型评价与结论

## 9.1 模型特点

本文以统一的有限体积方法处理常物性、变物性和实测收缩三种情形，得到题设位置的温度、含水率分布，并用全域最大含水率确定干燥终点。问题四中引入移动坐标后，变化的几何区域被映射到固定计算区间，同时保留了实际表面与固定物理位置的区别。网格加密、通量平衡和独立程序复算相互补充，为数值结果提供了检验。

## 9.2 局限与改进方向

模型仍有一定局限。预测时长取决于模型结构和输入数据的解释：一维中截面近似尚未量化端面影响，长期环境由前 4 h 观测延拓得到，气固边界采用有效浓度差并沿用题设表面系数；此外，模型未计入潜热，经验密度与收缩关系也未严格闭合。现有检验能够说明数值求解相互一致，但物理预测精度仍需独立的内部含水率观测验证。

后续工作可先补充长期烘房记录和内部温湿度测量，用于检验环境平台及干燥终点；再结合二维模型、吸附平衡关系和几何变形观测，进一步评估端面效应及质量闭合对预测结果的影响。

## 9.3 结论

综合四个问题可以看到，药材烘干先表现为表层快速失水，随后转为内部水分缓慢向外迁移；即使温度已经接近均匀，含水率仍可能高于达标阈值。在本文假设下，固定半径模型和实测收缩模型的临界时长分别为 57.4740 h 和 51.0920 h；若设备按整分钟停止，则分别取 57.4833 h 和 51.1000 h。相同物性下的固定几何对照表明，收缩是影响预测时长的重要因素。上述结果可为工艺估计提供参考，但实际应用仍需结合长期环境记录和内部含水率测量进行校验。

## AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型推导辅助、代码实现与调试、数值复算和结果核查、论文初稿组织及语言润色，详细使用情况见支撑材料。

参赛队对模型、代码、数值、参考文献和最终表述承担核验责任。工具的具体用途与核验方式见支撑材料中的《AI 工具使用详情.pdf》，本节披露不作为参考文献引用。

## 参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 A 题：药材的烘干问题及附件[Z]. 2026.
- [2] DA SILVA W P, E SILVA C M D P S, GAMA F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65-73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [3] ADROVER A, VENDITTI C, BRASIELLO A. A non-isothermal moving-boundary model for continuous and intermittent drying of pears[J]. Foods, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.

## 附录 A 支撑材料与复现说明

支撑材料 ZIP 内的文件列表如下。论文电子版从摘要页开始，不含承诺书和编号专用页。文中四位小数用于题设输出，终点判定始终使用未舍入结果。

AI 工具使用详情.pdf  
README\_复现说明.md  
\_\_init\_\_.py  
attachment1.csv  
attachment2.csv  
audit\_all\_results.py  
build\_result4.py  
compress\_problem2\_full\_process.py  
extend\_problem2\_delivery.py  
final\_all\_questions\_audit.json  
flat\_support\_audit.json  
problem1.py  
problem1\_radial\_profiles.png  
problem1\_time\_histories.png  
problem2.py  
problem2\_full\_process\_audit.json  
problem2\_radial\_profiles.png  
problem2\_time\_histories.png  
problem3.py  
problem3\_radial\_profiles.png  
problem3\_time\_histories.png  
problem4.py  
problem4\_independent.py  
problem4\_independent\_validation.json  
problem4\_production\_diagnostics.json  
problem4\_radial\_profiles.png  
problem4\_radius\_history.png  
problem4\_review\_experiments.json  
problem4\_time\_histories.png  
problem4\_workbook\_audit.json  
rebuild\_delivery\_workbooks.py  
requirements.txt  
restore\_full\_result2.py  
result1.xlsx  
result2.xlsx  
result2\_full\_process.npz  
result3.xlsx  
result4.xlsx  
run\_problem1.py  
run\_problem2.py  
run\_problem3.py  
run\_problem4.py  
test\_final\_review\_regressions.py  
test\_problem1.py  
test\_problem2.py  
test\_problem3.py  
test\_problem4.py  
test\_problem4\_independent.py  
test\_result4\_workbook.py  
verify\_problem4.py  
verify\_problem4\_independent.py

### A.1 运行顺序

先在支撑材料根目录安装 requirements.txt 中的依赖。Python 版本须为 3.11 或以上。支撑材料采用单层结构，按下面顺序运行即可：

```
python -m pip install -r requirements.txt
python run_problem1.py
python run_problem2.py
python run_problem3.py
python run_problem4.py
```

四个求解脚本在根目录产生 `problem1_result.json` 至 `problem4_result.json`，并生成相应图形；不会覆盖已填写的四份结果工作簿。复现所需 CSV 已包含在支撑材料中。

若仅需查看全程第二问 Excel，无须重新求解，在支撑材料根目录执行 `python restore_full_result2.py`，即可由压缩数组还原 `result2_full_process.xlsx`。

## A.2 全程输出与数值核验记录

第二问全程逐秒 Excel 约 27.7 MB。为满足支撑材料压缩包不超过 20 MB 的要求，包中以 `result2_full_process.npz` 无损保存四位小数交付值，并附 Excel 恢复脚本；`result2.xlsx` 为原前三小时文件。包外另提供的全程 Excel 与恢复文件的两张工作表 XML 逐字节相同。

全程积分沿用第二问的方程、初值和完整节点终态，输出不由舍入后的分钟值插值得到。其前 3 h 与原第二问交付值完全一致，与第三问独立分钟积分的最大四位小数差为 0.0001 kg/kg，差异来自积分分段与舍入边界。

原四份工作簿共回读 700974 个单元格，题设表 1—6 与对应数值输出一致；第二、三问原始分钟序列在前三小时重叠的 3780 个含水率值相同。第二问全程逐秒工作簿另有独立回读记录，见 `problem2_full_process_audit.json`。

支撑材料根目录保存原四问审计、第四问敏感性和密度诊断、全程逐秒结果审计。程序核验与参赛队人工审查是两类不同记录；AI 工具使用详情仅记录可确认的自动检查，不代替或虚构人工核验。

## 附录 B 核心源程序

下列仅列出问题一至问题四的核心模型代码。运行入口、独立验证、测试程序和工作簿工具等完整源码均直接放在支撑材料根目录，附录不重复展开。附录页数不计入正文限制。

### problem1.py

```
"""Problem 1 material laws and chamber boundary interpolation."""

from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import diags
```

```

@dataclass(frozen=True)
class ChamberHistory:
    """Observed chamber boundary conditions."""

    time_s: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        temperature = np.asarray(self.temperature_c, dtype=float)
        moisture = np.asarray(self.moisture_concentration, dtype=float)
        if time.ndim != 1 or temperature.shape != time.shape or moisture.shape != time.shape:
            raise ValueError("Chamber history columns must be one-dimensional and equal-sized")
        if any(np.any(~np.isfinite(values)) for values in (time, temperature, moisture)):
            raise ValueError("Chamber history must contain only finite values")
        if time.size < 2 or np.any(np.diff(time) <= 0.0):
            raise ValueError("Chamber times must be strictly increasing")
        if np.any(moisture < 0.0):
            raise ValueError("Chamber moisture concentration must be non-negative")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "temperature_c", temperature)
        object.__setattr__(self, "moisture_concentration", moisture)

@dataclass(frozen=True)
class Problem1Parameters:
    radius_m: float = 0.02
    initial_temperature_c: float = 28.0
    initial_moisture_concentration: float = 2.55
    density_kg_m3: float = 820.0
    heat_capacity_j_kg_k: float = 2600.0
    thermal_conductivity_w_m_k: float = 0.36
    heat_transfer_coefficient_w_m2_k: float = 25.0
    mass_transfer_coefficient_m_s: float = 8.0e-7

@dataclass(frozen=True)
class Problem1Solution:
    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

def load_chamber_history_csv(path: str | Path) -> ChamberHistory:
    """Load the three columns supplied in Attachment 1 from a UTF-8 CSV."""
    rows: list[tuple[float, float, float]] = []
    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        required = {"时间", "温度", "水分浓度"}
        if reader.fieldnames is None or not required.issubset(reader.fieldnames):
            raise ValueError("Attachment 1 must contain 时间, 温度, 水分浓度 columns")
        for line_number, row in enumerate(reader, start=2):
            try:
                rows.append((float(row["时间"]), float(row["温度"]), float(row["水分浓度"])))
            except (TypeError, ValueError) as exc:
                raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc
    if not rows:
        raise ValueError("Attachment 1 contains no data rows")
    data = np.asarray(rows, dtype=float)
    if not np.all(np.isfinite(data)):
        raise ValueError("Attachment 1 contains non-finite values")
    return ChamberHistory(data[:, 0], data[:, 1], data[:, 2])

def diffusivity_q1(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return  $D=7e-9 \exp(-0.89/C)$  in square metres per second."""
    values = np.asarray(concentration, dtype=float)
    if np.any(values <= 0.0):

```

```

        raise ValueError("Moisture concentration must be positive")
    return 7.0e-9 * np.exp(-0.89 / values)

def interpolate_history(
    query_time_s: float,
    time_s: ArrayLike,
    values: ArrayLike,
) -> float:
    """Linearly interpolate an observed chamber history."""
    return float(np.interp(query_time_s, time_s, values))

def nodal_control_volumes(node_count: int, radius_m: float) -> NDArray[np.float64]:
    """Return annular control volumes per unit cylinder length for nodal radii."""
    if node_count < 2:
        raise ValueError("At least two radial nodes are required")
    if radius_m <= 0.0:
        raise ValueError("Cylinder radius must be positive")
    spacing = radius_m / (node_count - 1)
    radii = np.linspace(0.0, radius_m, node_count)
    inner = np.maximum(radii - 0.5 * spacing, 0.0)
    outer = np.minimum(radii + 0.5 * spacing, radius_m)
    return np.pi * (outer**2 - inner**2)

def radial_flux_divergence(
    state: ArrayLike,
    coefficient: ArrayLike,
    radius_m: float,
    exchange_coefficient: float,
    ambient_value: float,
) -> NDArray[np.float64]:
    """Conservative cylindrical divergence with a convective outer boundary.

    ``coefficient`` is k for heat or D for moisture. At the outer surface the
    imposed flux is ``-exchange_coefficient * (surface - ambient)``.
    """
    values = np.asarray(state, dtype=float)
    conductance = np.asarray(coefficient, dtype=float)
    if values.ndim != 1 or conductance.shape != values.shape:
        raise ValueError("State and coefficient must be one-dimensional and equal-sized")
    if np.any(conductance < 0.0) or exchange_coefficient < 0.0:
        raise ValueError("Transport coefficients must be non-negative")

    node_count = values.size
    spacing = radius_m / (node_count - 1)
    face_radii = (np.arange(node_count - 1, dtype=float) + 0.5) * spacing
    denominator = conductance[:-1] + conductance[1:]
    face_coefficient = np.divide(
        2.0 * conductance[:-1] * conductance[1:],
        denominator,
        out=np.zeros_like(denominator),
        where=denominator > 0.0,
    )
    face_gradient = np.diff(values) / spacing
    face_rate = 2.0 * np.pi * face_radii * face_coefficient * face_gradient

    integrated_rate = np.zeros_like(values)
    integrated_rate[:-1] += face_rate
    integrated_rate[1:] -= face_rate
    surface_flux = -exchange_coefficient * (values[-1] - ambient_value)
    integrated_rate[-1] += 2.0 * np.pi * radius_m * surface_flux

    return integrated_rate / nodal_control_volumes(node_count, radius_m)

def solve_problem1(
    history: ChamberHistory,
    radial_intervals: int,

```

```

output_times_s: ArrayLike,
*,
parameters: Problem1Parameters = Problem1Parameters(),
relative_tolerance: float = 1.0e-8,
max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve the fixed-radius preheating model at requested output times."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    if output_times[0] < history.time_s[0] or output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times must lie within the chamber history")

    node_count = radial_intervals + 1
    radii = np.linspace(0.0, parameters.radius_m, node_count)
    sparsity = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1],
        shape=(node_count, node_count),
        format="csr",
    )

    thermal_coefficient = np.full(node_count, parameters.thermal_conductivity_w_m_k)

    def temperature_rhs(time_s: float, temperature_c: NDArray[np.float64]) -> NDArray[np.float64]:
        ambient = interpolate_history(time_s, history.time_s, history.temperature_c)
        divergence = radial_flux_divergence(
            temperature_c,
            thermal_coefficient,
            parameters.radius_m,
            parameters.heat_transfer_coefficient_w_m2_k,
            ambient,
        )
        return divergence / (parameters.density_kg_m3 * parameters.heat_capacity_j_kg_k)

    def log_moisture_rhs(time_s: float, log_concentration: NDArray[np.float64]) -> NDArray[np.float64]:
        concentration = np.exp(log_concentration)
        ambient = interpolate_history(time_s, history.time_s, history.moisture_concentration)
        divergence = radial_flux_divergence(
            concentration,
            diffusivity_q1(concentration),
            parameters.radius_m,
            parameters.mass_transfer_coefficient_m_s,
            ambient,
        )
        return divergence / concentration

    time_span = (float(history.time_s[0]), float(output_times[-1]))
    temperature_result = solve_ivp(
        temperature_rhs,
        time_span,
        np.full(node_count, parameters.initial_temperature_c),
        method="BDF",
        t_eval=output_times,
        rtol=relative_tolerance,
        atol=1.0e-9,
        max_step=max_step_s,
        jac_sparsity=sparsity,
    )
    moisture_result = solve_ivp(
        log_moisture_rhs,
        time_span,
        np.full(node_count, np.log(parameters.initial_moisture_concentration)),
        method="BDF",
        t_eval=output_times,
        rtol=relative_tolerance,

```

```

        atol=1.0e-10,
        max_step=max_step_s,
        jac_sparsity=sparsity,
    )
    if not temperature_result.success:
        raise RuntimeError(f"Temperature solver failed: {temperature_result.message}")
    if not moisture_result.success:
        raise RuntimeError(f"Moisture solver failed: {moisture_result.message}")

    return Problem1Solution(
        time_s=output_times,
        radius_m=radii,
        temperature_c=temperature_result.y.T,
        moisture_concentration=np.exp(moisture_result.y.T),
    )

def sample_solution(solution: Problem1Solution, radius_cm: ArrayLike) -> Problem1Solution:
    """Interpolate a solution to requested physical radii in centimetres."""
    requested_radius_m = np.asarray(radius_cm, dtype=float) * 0.01
    if requested_radius_m.ndim != 1 or requested_radius_m.size == 0:
        raise ValueError("Requested radii must be a non-empty one-dimensional array")
    if np.any(np.diff(requested_radius_m) <= 0.0):
        raise ValueError("Requested radii must be strictly increasing")
    if requested_radius_m[0] < 0.0 or requested_radius_m[-1] > solution.radius_m[-1]:
        raise ValueError("Requested radii lie outside the cylinder")

    temperature = np.vstack(
        [np.interp(requested_radius_m, solution.radius_m, row) for row in solution.temperature_c]
    )
    moisture = np.vstack(
        [
            np.interp(requested_radius_m, solution.radius_m, row)
            for row in solution.moisture_concentration
        ]
    )
    return Problem1Solution(
        time_s=solution.time_s.copy(),
        radius_m=requested_radius_m,
        temperature_c=temperature,
        moisture_concentration=moisture,
    )

def richardson_extrapolate_solutions(
    coarse: Problem1Solution,
    fine: Problem1Solution,
    *,
    order: int = 2,
) -> Problem1Solution:
    """Extrapolate two solutions whose spatial mesh widths differ by a factor of two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if (
        coarse.temperature_c.shape != fine.temperature_c.shape
        or coarse.moisture_concentration.shape
        != fine.moisture_concentration.shape
    ):
        raise ValueError("Solutions must have matching field shapes")
    if not np.array_equal(coarse.time_s, fine.time_s) or not np.array_equal(
        coarse.radius_m,
        fine.radius_m,
    ):
        raise ValueError("Solutions must use identical output times and radii")
    denominator = 2.0**order - 1.0
    return Problem1Solution(
        time_s=fine.time_s.copy(),
        radius_m=fine.radius_m.copy(),
        temperature_c=fine.temperature_c
        + (fine.temperature_c - coarse.temperature_c) / denominator,

```

```

        moisture_concentration=fine.moisture_concentration
        + (fine.moisture_concentration - coarse.moisture_concentration) / denominator,
    )

def result_payload(solution: Problem1Solution) -> dict[str, list]:
    """Convert a sampled solution to the numeric schema used by result1.xlsx."""
    if np.any(~np.isfinite(solution.time_s)) or not np.allclose(solution.time_s,
np.round(solution.time_s), atol=1.0e-10, rtol=0.0):
        raise ValueError("Workbook output times must be whole seconds")
    return {
        "time_s": np.round(solution.time_s).astype(int).tolist(),
        "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
        "temperature_c": np.round(solution.temperature_c, 4).tolist(),
        "moisture_concentration": np.round(solution.moisture_concentration, 4).tolist(),
    }

```

## problem2.py

```

"""Coupled variable-property model for Problem 2."""

from __future__ import annotations

from collections.abc import Callable

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import bmat, diags

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    interpolate_history,
    radial_flux_divergence,
    sample_solution,
)

ScalarLaw = Callable[[NDArray[np.float64]], NDArray[np.float64]]
CoupledLaw = Callable[
    [NDArray[np.float64], NDArray[np.float64]],
    NDArray[np.float64],
]

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:
    values = np.asarray(concentration, dtype=float)
    if np.any(~np.isfinite(values)) or np.any(values < 0.0):
        raise ValueError("Moisture concentration must be finite and non-negative")
    return values

def density_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 density in kilograms per cubic metre."""
    values = _concentration_array(concentration)
    return 650.0 + 128.0 * values

def heat_capacity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 specific heat capacity in joules per kilogram-kelvin."""
    values = _concentration_array(concentration)
    return 1450.0 + 2736.0 * values / (values + 1.0)

def thermal_conductivity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 thermal conductivity in watts per metre-kelvin."""
    values = _concentration_array(concentration)

```

```

        return 0.21 + 0.38 * values / (values + 1.0)

def diffusivity_q2(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:
    """Return Appendix 3 moisture diffusivity using absolute temperature."""
    moisture = _concentration_array(concentration)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
        raise ValueError("Temperature must be finite and above absolute zero")
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must be positive for diffusivity")
    moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
    return 2.4e-3 * np.exp(-0.45 / moisture) * np.exp(-3850.0 / temperature_k)

def solve_variable_property_fixed_cylinder(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    density_law: ScalarLaw,
    heat_capacity_law: ScalarLaw,
    conductivity_law: ScalarLaw,
    diffusivity_law: CoupledLaw,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve coupled heat and moisture transport on a fixed cylindrical radius."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if (
        not np.isfinite(start_time)
        or start_time < history.time_s[0]
        or start_time > output_times[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if maximum_moisture_threshold is not None and (
        not np.isfinite(maximum_moisture_threshold)
        or maximum_moisture_threshold <= 0.0
        or maximum_moisture_threshold >= parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

    node_count = radial_intervals + 1
    radii = np.linspace(0.0, parameters.radius_m, node_count)
    tridiagonal = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1],
        shape=(node_count, node_count),
        format="csr",
    )
    sparsity = bmat(
        [[tridiagonal, tridiagonal], [tridiagonal, tridiagonal]],
        format="csr",
    )

```

```

def coupled_rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
    temperature_c = state[:node_count]
    concentration = np.exp(state[node_count:])
    density = np.asarray(density_law(concentration), dtype=float)
    heat_capacity = np.asarray(heat_capacity_law(concentration), dtype=float)
    conductivity = np.asarray(conductivity_law(concentration), dtype=float)
    diffusivity = np.asarray(
        diffusivity_law(concentration, temperature_c),
        dtype=float,
    )
    expected_shape = concentration.shape
    for name, values in (
        ("density", density),
        ("heat capacity", heat_capacity),
        ("thermal conductivity", conductivity),
        ("diffusivity", diffusivity),
    ):
        if values.shape != expected_shape or np.any(~np.isfinite(values)) or np.any(values <= 0.0):
            raise ValueError(f"{name} law must return finite positive nodal values")

    ambient_temperature = interpolate_history(
        time_s,
        history.time_s,
        history.temperature_c,
    )
    temperature_rate = radial_flux_divergence(
        temperature_c,
        conductivity,
        parameters.radius_m,
        parameters.heat_transfer_coefficient_w_m2_k,
        ambient_temperature,
    ) / (density * heat_capacity)

    ambient_moisture = interpolate_history(
        time_s,
        history.time_s,
        history.moisture_concentration,
    )
    concentration_rate = radial_flux_divergence(
        concentration,
        diffusivity,
        parameters.radius_m,
        parameters.mass_transfer_coefficient_m_s,
        ambient_moisture,
    )
    log_concentration_rate = concentration_rate / concentration
    return np.concatenate([temperature_rate, log_concentration_rate])

supplied_initial_fields = (
    initial_temperature_c is not None,
    initial_moisture_concentration is not None,
)
if any(supplied_initial_fields) and not all(supplied_initial_fields):
    raise ValueError("Both initial fields must be supplied together")
if all(supplied_initial_fields):
    temperature_initial = np.asarray(initial_temperature_c, dtype=float)
    moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)
    if (
        temperature_initial.shape != (node_count,)
        or moisture_initial.shape != (node_count,)
        or np.any(~np.isfinite(temperature_initial))
        or np.any(~np.isfinite(moisture_initial))
        or np.any(moisture_initial <= 0.0)
    ):
        raise ValueError("Initial fields must be finite positive nodal arrays")
else:
    temperature_initial = np.full(node_count, parameters.initial_temperature_c)
    moisture_initial = np.full(
        node_count,

```

```

        parameters.initial_moisture_concentration,
    )
    initial_state = np.concatenate(
        [temperature_initial, np.log(moisture_initial)]
    )
    absolute_tolerance = np.concatenate(
        [np.full(node_count, 1.0e-9), np.full(node_count, 1.0e-10)]
    )

    threshold_event = None
    if maximum_moisture_threshold is not None:

        def threshold_event(time_s: float, state: NDArray[np.float64]) -> float:
            del time_s
            return float(
                np.max(np.exp(state[node_count:])) - maximum_moisture_threshold
            )

        threshold_event.terminal = True
        threshold_event.direction = -1.0

    result = solve_ivp(
        coupled_rhs,
        (start_time, float(output_times[-1])),
        initial_state,
        method="BDF",
        t_eval=output_times,
        rtol=relative_tolerance,
        atol=absolute_tolerance,
        max_step=max_step_s,
        jac_sparsity=sparsity,
        events=threshold_event,
    )
    if not result.success:
        raise RuntimeError(f"Coupled solver failed: {result.message}")
    result_time = result.t
    result_state = result.y
    if maximum_moisture_threshold is not None:
        if not result.t_events or result.t_events[0].size == 0:
            raise RuntimeError("Maximum moisture threshold was not reached")
        event_time = float(result.t_events[0][0])
        event_state = result.y_events[0][0]
        if result_time.size and np.isclose(
            result_time[-1],
            event_time,
            rtol=0.0,
            atol=1.0e-9,
        ):
            result_time = result_time.copy()
            result_state = result_state.copy()
            result_time[-1] = event_time
            result_state[:, -1] = event_state
        else:
            result_time = np.append(result_time, event_time)
            result_state = np.column_stack([result_state, event_state])
    return Problem1Solution(
        time_s=result_time,
        radius_m=radii,
        temperature_c=result_state[:node_count].T,
        moisture_concentration=np.exp(result_state[node_count:].T),
    )

def solve_problem2(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,

```

```

max_step_s: float = 5.0,
maximum_moisture_threshold: float | None = None,
initial_time_s: float | None = None,
initial_temperature_c: ArrayLike | None = None,
initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve Problem 2 with the four Appendix 3 material-property laws."""
    return solve_variable_property_fixed_cylinder(
        history,
        radial_intervals,
        output_times_s,
        density_law=density_q2,
        heat_capacity_law=heat_capacity_q2,
        conductivity_law=thermal_conductivity_q2,
        diffusivity_law=diffusivity_q2,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=maximum_moisture_threshold,
        initial_time_s=initial_time_s,
        initial_temperature_c=initial_temperature_c,
        initial_moisture_concentration=initial_moisture_concentration,
    )

def solve_problem2_sampled_in_chunks(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    sample_radius_cm: ArrayLike,
    *,
    chunk_duration_s: float = 600.0,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve Problem 2 in bounded-memory chunks and retain sampled radii only."""
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(~np.isfinite(output_times)) or np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be finite and strictly increasing")
    if (
        output_times[0] < history.time_s[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if not np.isfinite(chunk_duration_s) or chunk_duration_s <= 0.0:
        raise ValueError("Chunk duration must be finite and positive")

    requested_radius_cm = np.asarray(sample_radius_cm, dtype=float)
    node_count = radial_intervals + 1
    nodal_radius = np.linspace(0.0, parameters.radius_m, node_count)
    initial_solution = Problem1Solution(
        time_s=np.array([history.time_s[0]]),
        radius_m=nodal_radius,
        temperature_c=np.full((1, node_count), parameters.initial_temperature_c),
        moisture_concentration=np.full(
            (1, node_count),
            parameters.initial_moisture_concentration,
        ),
    )
    sampled_initial = sample_solution(initial_solution, requested_radius_cm)
    sampled_temperature = np.empty(
        (output_times.size, sampled_initial.radius_m.size),
        dtype=float,
    )
    sampled_moisture = np.empty_like(sampled_temperature)

    next_output_index = 0

```

```

current_time = float(history.time_s[0])
current_temperature: NDArray[np.float64] | None = None
current_moisture: NDArray[np.float64] | None = None
if np.isclose(output_times[0], current_time, rtol=0.0, atol=1.0e-12):
    sampled_temperature[0] = sampled_initial.temperature_c[0]
    sampled_moisture[0] = sampled_initial.moisture_concentration[0]
    next_output_index = 1

final_time = float(output_times[-1])
while current_time < final_time:
    chunk_end = min(current_time + chunk_duration_s, final_time)
    chunk_stop = int(np.searchsorted(output_times, chunk_end, side="right"))
    requested_times = output_times[next_output_index:chunk_stop]
    if requested_times.size and requested_times[0] <= current_time:
        raise RuntimeError("Chunked output indexing did not advance")
    append_chunk_end = (
        requested_times.size == 0
        or not np.isclose(requested_times[-1], chunk_end, rtol=0.0, atol=1.0e-12)
    )
    integration_times = (
        np.append(requested_times, chunk_end)
        if append_chunk_end
        else requested_times
    )
    chunk_solution = solve_problem2(
        history,
        radial_intervals=radial_intervals,
        output_times_s=integration_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        initial_time_s=None if current_temperature is None else current_time,
        initial_temperature_c=current_temperature,
        initial_moisture_concentration=current_moisture,
    )
    sampled_chunk = sample_solution(chunk_solution, requested_radius_cm)
    retained_count = requested_times.size
    if retained_count:
        sampled_temperature[next_output_index:chunk_stop] = (
            sampled_chunk.temperature_c[:retained_count]
        )
        sampled_moisture[next_output_index:chunk_stop] = (
            sampled_chunk.moisture_concentration[:retained_count]
        )
    current_temperature = chunk_solution.temperature_c[-1].copy()
    current_moisture = chunk_solution.moisture_concentration[-1].copy()
    current_time = chunk_end
    next_output_index = chunk_stop

if next_output_index != output_times.size:
    raise RuntimeError("Chunked solver did not populate every requested output time")
return Problem1Solution(
    time_s=output_times.copy(),
    radius_m=sampled_initial.radius_m,
    temperature_c=sampled_temperature,
    moisture_concentration=sampled_moisture,
)

```

## problem3.py

```

"""Fixed-radius drying endpoint model for Problem 3."""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

import numpy as np

```

```

from scipy.integrate import cumulative_simpson

```

```

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    nodal_control_volumes,
)
from problem2 import solve_problem2

@dataclass(frozen=True)
class Problem3Result:
    """Problem 3 solution together with its unrounded drying endpoint."""

    solution: Problem1Solution
    drying_time_s: float
    strict_completion_time_s: float | None
    maximum_moisture: float
    maximum_radius_m: float
    plateau_temperature_c: float
    plateau_moisture_concentration: float
    threshold: float
    output_interval_s: float

def extend_chamber_history_to_plateau(
    history: ChamberHistory,
    end_time_s: float,
    *,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,
) -> ChamberHistory:
    """Extend Attachment 1 with a short transition to a constant tail plateau."""
    if not np.isfinite(end_time_s) or end_time_s <= history.time_s[-1]:
        raise ValueError("Extension end time must be later than the observed history")
    if not np.isfinite(averaging_window_s) or averaging_window_s <= 0.0:
        raise ValueError("Averaging window must be positive")
    if not np.isfinite(transition_s) or transition_s <= 0.0:
        raise ValueError("Plateau transition must be positive")

    tail = history.time_s >= history.time_s[-1] - averaging_window_s
    if np.count_nonzero(tail) < 2:
        raise ValueError("Averaging window must contain at least two observations")
    temperature = (
        float(np.mean(history.temperature_c[tail]))
        if plateau_temperature_c is None
        else float(plateau_temperature_c)
    )
    moisture = (
        float(np.mean(history.moisture_concentration[tail]))
        if plateau_moisture_concentration is None
        else float(plateau_moisture_concentration)
    )
    if not np.isfinite(temperature):
        raise ValueError("Plateau temperature must be finite")
    if not np.isfinite(moisture) or moisture < 0.0:
        raise ValueError("Plateau moisture concentration must be finite and non-negative")

    transition_time = min(history.time_s[-1] + transition_s, end_time_s)
    appended_times = [transition_time]
    if end_time_s > transition_time:
        appended_times.append(end_time_s)
    return ChamberHistory(
        time_s=np.concatenate([history.time_s, np.asarray(appended_times)]),
        temperature_c=np.concatenate(
            [history.temperature_c, np.full(len(appended_times), temperature)]
        ),
        moisture_concentration=np.concatenate(

```

```

        [history.moisture_concentration, np.full(len(appended_times), moisture)]
    ),
)

def solve_problem3(
    history: ChamberHistory,
    radial_intervals: int,
    horizon_s: float,
    *,
    output_interval_s: float = 60.0,
    threshold: float = 0.15,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 60.0,
) -> Problem3Result:
    """Continue the Problem 2 model until the full field reaches the threshold."""
    if not np.isfinite(output_interval_s) or output_interval_s <= 0.0:
        raise ValueError("Output interval must be positive")
    extended = extend_chamber_history_to_plateau(
        history,
        horizon_s,
        averaging_window_s=averaging_window_s,
        transition_s=transition_s,
        plateau_temperature_c=plateau_temperature_c,
        plateau_moisture_concentration=plateau_moisture_concentration,
    )
    regular_times = np.arange(
        output_interval_s,
        horizon_s + 0.5 * output_interval_s,
        output_interval_s,
    )
    regular_times = regular_times[regular_times <= horizon_s]
    if regular_times.size == 0 or regular_times[-1] < horizon_s:
        output_times = np.append(regular_times, horizon_s)
    else:
        output_times = regular_times
    solution = solve_problem2(
        extended,
        radial_intervals,
        output_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=threshold,
    )
    terminal_moisture = solution.moisture_concentration[-1]
    maximum_index = int(np.argmax(terminal_moisture))
    return Problem3Result(
        solution=solution,
        drying_time_s=float(solution.time_s[-1]),
        strict_completion_time_s=None,
        maximum_moisture=float(terminal_moisture[maximum_index]),
        maximum_radius_m=float(solution.radius_m[maximum_index]),
        plateau_temperature_c=float(extended.temperature_c[-1]),
        plateau_moisture_concentration=float(extended.moisture_concentration[-1]),
        threshold=float(threshold),
        output_interval_s=float(output_interval_s),
    )

def richardson_extrapolate_drying_time(
    coarse_time_s: float,
    fine_time_s: float,
    *,
    order: int = 2,

```

```

) -> float:
    """Extrapolate drying times from meshes whose spacing ratio is two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if not np.isfinite(coarse_time_s) or not np.isfinite(fine_time_s):
        raise ValueError("Drying times must be finite")
    return float(fine_time_s + (fine_time_s - coarse_time_s) / (2.0*order - 1.0))

def validate_radially_nonincreasing_moisture(
    solution: Problem1Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require every stored profile to be wettest at the centre and nonincreasing."""
    if not np.isfinite(tolerance) or tolerance < 0.0:
        raise ValueError("Radial monotonicity tolerance must be finite and non-negative")
    if solution.radius_m.size < 2:
        raise ValueError("Radial monotonicity requires at least two positions")
    maximum_outward_increase = float(
        np.max(np.diff(solution.moisture_concentration, axis=1))
    )
    if maximum_outward_increase > tolerance:
        raise ValueError(
            "Moisture profile contains an outward increase; centre-wettest claim failed"
        )
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem1Solution,
    history: ChamberHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Compare total moisture change with integrated convective surface flux."""
    if not np.isclose(solution.time_s[0], history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time solution row")
    if solution.time_s[-1] > history.time_s[-1]:
        raise ValueError("Moisture balance times must lie within the chamber history")
    if not np.isclose(solution.radius_m[-1], parameters.radius_m, atol=1.0e-12, rtol=0.0):
        raise ValueError("Solution radius and model radius must match")

    volumes = nodal_control_volumes(solution.radius_m.size, parameters.radius_m)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient_moisture = np.interp(
        solution.time_s,
        history.time_s,
        history.moisture_concentration,
    )
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / parameters.radius_m
        * (solution.moisture_concentration[:, -1] - ambient_moisture)
    )
    cumulative_surface_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,
        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - cumulative_surface_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(
            mean_moisture[-1] - mean_moisture[0]
        )
    }

```

```

    ),
    "integrated_surface_change": float(cumulative_surface_change[-1]),
}

def truncate_solution_at_threshold(
    solution: Problem1Solution,
    *,
    threshold: float,
    output_interval_s: float,
) -> Problem1Solution:
    """Insert the threshold crossing and retain the first later strict regular row."""
    maximum = np.max(solution.moisture_concentration, axis=1)
    crossing_indices = np.flatnonzero(maximum <= threshold)
    if crossing_indices.size == 0 or crossing_indices[0] == 0:
        raise ValueError("Solution must bracket the first moisture threshold crossing")
    upper = int(crossing_indices[0])
    lower = upper - 1
    # Interpolate each nodal field, then take the last nodal crossing. Merely
    # interpolating the two maxima is wrong when the wettest node changes.
    above = solution.moisture_concentration[lower] > threshold
    nodal_drop = solution.moisture_concentration[lower, above] - solution.moisture_concentration[upper,
above]
    fraction = float(np.max((solution.moisture_concentration[lower, above] - threshold) / nodal_drop))
    crossing_time = solution.time_s[lower] + fraction * (
        solution.time_s[upper] - solution.time_s[lower]
    )
    crossing_temperature = solution.temperature_c[lower] + fraction * (
        solution.temperature_c[upper] - solution.temperature_c[lower]
    )
    crossing_moisture = solution.moisture_concentration[lower] + fraction * (
        solution.moisture_concentration[upper]
        - solution.moisture_concentration[lower]
    )
    on_regular_interval = np.isclose(
        np.mod(solution.time_s, output_interval_s),
        0.0,
        atol=1.0e-8,
    )
    regular_before = on_regular_interval & (
        solution.time_s < crossing_time - 1.0e-8
    )
    strict_candidates = np.flatnonzero(
        on_regular_interval
        & (solution.time_s > crossing_time + 1.0e-8)
        & (maximum < threshold)
    )
    if strict_candidates.size == 0:
        raise ValueError("Solution must include a later regular row strictly below threshold")
    strict_index = int(strict_candidates[0])
    return Problem1Solution(
        time_s=np.concatenate(
            [solution.time_s[regular_before], [crossing_time, solution.time_s[strict_index]]]
        ),
        radius_m=solution.radius_m.copy(),
        temperature_c=np.vstack(
            [
                solution.temperature_c[regular_before],
                crossing_temperature,
                solution.temperature_c[strict_index],
            ]
        ),
        moisture_concentration=np.vstack(
            [
                solution.moisture_concentration[regular_before],
                crossing_moisture,
                solution.moisture_concentration[strict_index],
            ]
        ),
    ),

```

```

)

def problem3_payload(result: Problem3Result) -> dict[str, object]:
    """Convert a sampled Problem 3 result to the result3.xlsx data schema."""
    solution = result.solution
    if solution.time_s.size == 0 or np.any(np.diff(solution.time_s) <= 0.0):
        raise ValueError("Problem 3 output times must be non-empty and increasing")
    if result.strict_completion_time_s is None:
        raise ValueError("Strict completion time requires a verified post-threshold row")
    if not np.isclose(
        solution.time_s[-1],
        result.strict_completion_time_s,
        rtol=0.0,
        atol=1.0e-8,
    ):
        raise ValueError("The final solution row must be the strict completion time")
    critical_indices = np.flatnonzero(
        np.isclose(
            solution.time_s,
            result.drying_time_s,
            rtol=0.0,
            atol=1.0e-8,
        )
    )
    if critical_indices.size != 1:
        raise ValueError("The solution must contain exactly one critical threshold row")
    critical_index = int(critical_indices[0])
    regular_rows = np.ones(solution.time_s.size, dtype=bool)
    regular_rows[critical_index] = False
    if not np.allclose(
        np.mod(solution.time_s[regular_rows], result.output_interval_s),
        0.0,
        atol=1.0e-8,
    ):
        raise ValueError("All non-critical output rows must lie on the regular interval")
    critical_maximum = float(
        np.max(solution.moisture_concentration[critical_index])
    )
    strict_maximum = float(np.max(solution.moisture_concentration[-1]))
    if not np.isclose(critical_maximum, result.threshold, atol=1.0e-8, rtol=0.0):
        raise ValueError("Critical maximum moisture must equal the unrounded threshold")
    if not strict_maximum < result.threshold:
        raise ValueError("Strict completion maximum moisture must be below threshold")

    time_values: list[int | float] = []
    for value in solution.time_s:
        rounded_integer = round(float(value))
        if np.isclose(value, rounded_integer, rtol=0.0, atol=1.0e-9):
            time_values.append(int(rounded_integer))
        else:
            time_values.append(round(float(value), 6))
    return {
        "time_s": time_values,
        "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
        "moisture_concentration": np.round(
            solution.moisture_concentration,
            4,
        ).tolist(),
        "drying_time_s": round(result.drying_time_s, 6),
        "drying_time_h": round(result.drying_time_s / 3600.0, 10),
        "strict_completion_time_s": round(result.strict_completion_time_s, 6),
        "strict_completion_time_h": round(
            result.strict_completion_time_s / 3600.0,
            10,
        ),
        "threshold": result.threshold,
        "critical_maximum_moisture_unrounded": critical_maximum,
        "strict_completion_maximum_moisture_unrounded": strict_maximum,
        "maximum_radius_m": result.maximum_radius_m,
    }

```

```

        "plateau_temperature_c": result.plateau_temperature_c,
        "plateau_moisture_concentration": result.plateau_moisture_concentration,
    }

```

## problem4.py

```

"""Moving-radius drying model for Problem 4.

```

```

The measured radius history is used as a prescribed moving boundary. The
radial coordinate is mapped to ``xi=r/R(t)``; for dry-basis concentration the
solid motion then cancels the grid velocity and leaves a diffusion equation on
the fixed interval ``0 <= xi <= 1`` under spatially uniform dry-solid density.
Appendix 4 density is used as an effective thermal-storage law; compatibility
with a literal wet-bulk-density interpretation is audited separately.
"""

```

```

from __future__ import annotations

```

```

import csv
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import cumulative_simpson, solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import bmat, diags

```

```

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    interpolate_history,
    nodal_control_volumes,
    radial_flux_divergence,
)

```

```

@dataclass(frozen=True)

```

```

class RadiusHistory:

```

```

    """Measured radius history in seconds and centimetres converted to metres."""

```

```

    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    interpolation: str = "linear"
    _pchip: PchipInterpolator | None = field(init=False, repr=False, compare=False)

```

```

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        radius = np.asarray(self.radius_m, dtype=float)
        if time.ndim != 1 or radius.shape != time.shape or time.size < 2:
            raise ValueError("Radius history columns must be one-dimensional and equal-sized")
        if np.any(~np.isfinite(time)) or np.any(~np.isfinite(radius)):
            raise ValueError("Radius history must be finite")
        if np.any(np.diff(time) <= 0.0):
            raise ValueError("Radius times must be strictly increasing")
        if np.any(radius <= 0.0) or np.any(np.diff(radius) > 1.0e-12):
            raise ValueError("Radius must remain positive and non-increasing")
        if self.interpolation not in ("linear", "pchip"):
            raise ValueError("Radius interpolation must be linear or pchip")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "radius_m", radius)
        object.__setattr__(self, "_pchip", PchipInterpolator(time, radius) if self.interpolation ==
"pchip" else None)

```

```

@dataclass(frozen=True)

```

```

class Problem4Solution:

```

```

    """Full solution on the dimensionless moving-boundary coordinate."""

```

```

time_s: NDArray[np.float64]
xi: NDArray[np.float64]
temperature_c: NDArray[np.float64]
moisture_concentration: NDArray[np.float64]

def load_radius_history_csv(path: str | Path) -> RadiusHistory:
    """Load Attachment 2 exported as UTF-8 CSV."""
    rows: list[tuple[float, float]] = []
    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        if reader.fieldnames is None:
            raise ValueError("Attachment 2 has no header")
        names = set(reader.fieldnames)
        time_name = next((name for name in ("时间", "time", "time_s", "t") if name in names), None)
        radius_name = next((name for name in ("半径", "radius", "radius_cm", "R") if name in names),
None)
        # The official workbook may be exported with a locale-specific or
        # mojibake header. Attachment 2 has exactly two columns, so a
        # positional fallback is unambiguous and keeps the data auditable.
        if time_name is None or radius_name is None:
            time_name, radius_name = reader.fieldnames[:2]
        for line_number, row in enumerate(reader, start=2):
            try:
                rows.append((float(row[time_name]), float(row[radius_name])))
            except (TypeError, ValueError) as exc:
                raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc
    if not rows:
        raise ValueError("Attachment 2 contains no data rows")
    values = np.asarray(rows, dtype=float)
    return RadiusHistory(values[:, 0], values[:, 1] * 0.01)

def interpolate_radius(time_s: ArrayLike, history: RadiusHistory) -> NDArray[np.float64]:
    """Interpolate radius with the selected method and hold the measured tail."""
    query = np.asarray(time_s, dtype=float)
    if np.any(~np.isfinite(query)) or np.any(query < history.time_s[0]):
        raise ValueError("Requested radius time precedes Attachment 2")
    if history._pchip is not None:
        return history._pchip(np.minimum(query, history.time_s[-1]))
    return np.interp(query, history.time_s, history.radius_m)

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:
    values = np.asarray(concentration, dtype=float)
    if np.any(~np.isfinite(values)) or np.any(values < 0.0):
        raise ValueError("Moisture concentration must be finite and non-negative")
    return values

def density_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 760.0 + 90.0 * values

def heat_capacity_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 1850.0 + 2150.0 * values / (values + 1.0)

def thermal_conductivity_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 0.12 + 0.20 * values / (values + 1.0)

def diffusivity_q4(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:

```

```

moisture = _concentration_array(concentration)
temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
    raise ValueError("Temperature must be finite and above absolute zero")
if np.any(moisture <= 0.0):
    raise ValueError("Moisture concentration must be positive for diffusivity")
moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
return 4.2e-4 * np.exp(-0.30 / moisture) * np.exp(-3850.0 / temperature_k)

def solve_problem4(
    history: ChamberHistory,
    radius_history: RadiusHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 2.0e-8,
    max_step_s: float = 60.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem4Solution:
    """Integrate the coupled Appendix 4 equations on ``xi=r/R(t)``."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0 or np.any(~np.isfinite(output_times)):
        raise ValueError("Output times must be a non-empty finite array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if not np.isfinite(start_time) or start_time < max(history.time_s[0], radius_history.time_s[0]) or
    start_time > output_times[0]:
        raise ValueError("Initial time must not be after the first output time")
    if output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times exceed the supplied chamber history")
    if maximum_moisture_threshold is not None and not (
        0.0 < maximum_moisture_threshold < parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

    node_count = radial_intervals + 1
    xi = np.linspace(0.0, 1.0, node_count)
    tri = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1], shape=(node_count, node_count), format="csr",
    )
    sparsity = bmat([[tri, tri], [tri, tri]], format="csr")

    supplied_initial = (initial_temperature_c is not None, initial_moisture_concentration is not None)
    if any(supplied_initial) and not all(supplied_initial):
        raise ValueError("Both initial fields must be supplied together")
    if all(supplied_initial):
        temperature_initial = np.asarray(initial_temperature_c, dtype=float)
        moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)
        if temperature_initial.shape != (node_count,) or moisture_initial.shape != (node_count,):
            raise ValueError("Initial fields have the wrong nodal shape")
        if np.any(~np.isfinite(temperature_initial)) or np.any(~np.isfinite(moisture_initial)) or
        np.any(moisture_initial <= 0.0):
            raise ValueError("Initial fields must be finite and moisture must be positive")
    else:
        temperature_initial = np.full(node_count, parameters.initial_temperature_c)
        moisture_initial = np.full(node_count, parameters.initial_moisture_concentration)

    def rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
        temperature = state[:node_count]
        concentration = np.exp(state[node_count:])
        radius = float(interpolate_radius(time_s, radius_history))

```

```

density = density_q4(concentration)
heat_capacity = heat_capacity_q4(concentration)
conductivity = thermal_conductivity_q4(concentration)
diffusivity = diffusivity_q4(concentration, temperature)
ambient_temperature = interpolate_history(time_s, history.time_s, history.temperature_c)
ambient_moisture = interpolate_history(time_s, history.time_s, history.moisture_concentration)
temperature_div = radial_flux_divergence(
    temperature, conductivity, 1.0, radius * parameters.heat_transfer_coefficient_w_m2_k,
    ambient_temperature,
) / radius**2
moisture_div = radial_flux_divergence(
    concentration, diffusivity, 1.0, radius * parameters.mass_transfer_coefficient_m_s,
    ambient_moisture,
) / radius**2
return np.concatenate([temperature_div / (density * heat_capacity), moisture_div /
concentration])

initial_state = np.concatenate([temperature_initial, np.log(moisture_initial)])
atol = np.concatenate([np.full(node_count, 1.0e-8), np.full(node_count, 1.0e-10)])
event = None
if maximum_moisture_threshold is not None:
    def event(time_s: float, state: NDArray[np.float64]) -> float:
        del time_s
        return float(np.max(np.exp(state[node_count:])) - maximum_moisture_threshold)
    event.terminal = True
    event.direction = -1.0

result = solve_ivp(
    rhs, (start_time, float(output_times[-1])), initial_state, method="BDF", t_eval=output_times,
    rtol=relative_tolerance, atol=atol, max_step=max_step_s, jac_sparsity=sparsity, events=event,
)
if not result.success:
    raise RuntimeError(f"Problem 4 solver failed: {result.message}")
result_time = result.t
result_state = result.y
if maximum_moisture_threshold is not None:
    if not result.t_events or result.t_events[0].size == 0:
        raise RuntimeError("Maximum moisture threshold was not reached")
    event_time = float(result.t_events[0][0])
    event_state = result.y_events[0][0]
    if result_time.size and np.isclose(result_time[-1], event_time, atol=1.0e-9, rtol=0.0):
        result_time = result_time.copy(); result_state = result_state.copy()
        result_time[-1] = event_time; result_state[:, -1] = event_state
    else:
        result_time = np.append(result_time, event_time)
        result_state = np.column_stack([result_state, event_state])
return Problem4Solution(
    time_s=np.asarray(result_time), xi=xi,
    temperature_c=result_state[:node_count].T,
    moisture_concentration=np.exp(result_state[node_count:].T),
)

def sample_moving_solution(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
    radius_cm: ArrayLike,
) -> NDArray[np.float64]:
    """Sample moisture at fixed physical radii; outside cells are NaN."""
    requested = np.asarray(radius_cm, dtype=float)
    if requested.ndim != 1 or requested.size == 0 or np.any(~np.isfinite(requested)) or
np.any(requested < 0.0) or np.any(np.diff(requested) <= 0.0):
        raise ValueError("Requested radii must be strictly increasing")
    radii_m = requested * 0.01
    sampled = np.full((solution.time_s.size, requested.size), np.nan, dtype=float)
    current_radius = interpolate_radius(solution.time_s, radius_history)
    for row, radius in enumerate(current_radius):
        valid = radii_m <= radius + 1.0e-12
        if np.any(valid):
            coordinates = radii_m[valid] / radius

```

```

        sampled[row, valid] = np.interp(coordinates, solution.xi,
solution.moisture_concentration[row])
        return sampled

def sample_surface(solution: Problem4Solution) -> NDArray[np.float64]:
    """Return the actual moving-surface concentration."""
    return solution.moisture_concentration[:, -1].copy()

def validate_radially_nonincreasing_moisture(
    solution: Problem4Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require the stored moving-coordinate profiles to decrease outward."""
    if not np.isfinite(tolerance) or tolerance < 0.0:
        raise ValueError("Tolerance must be finite and non-negative")
    maximum_outward_increase = float(np.max(np.diff(solution.moisture_concentration, axis=1)))
    if maximum_outward_increase > tolerance:
        raise ValueError("Moisture profile contains an outward increase")
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem4Solution,
    chamber_history: ChamberHistory,
    radius_history: RadiusHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Check normalized-domain moisture change against the moving surface flux."""
    if not np.isclose(solution.time_s[0], chamber_history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time row")
    volumes = nodal_control_volumes(solution.xi.size, 1.0)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient = np.interp(
        solution.time_s,
        chamber_history.time_s,
        chamber_history.moisture_concentration,
    )
    current_radius = interpolate_radius(solution.time_s, radius_history)
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / current_radius
        * (solution.moisture_concentration[:, -1] - ambient)
    )
    integrated_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,
        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - integrated_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(mean_moisture[-1] - mean_moisture[0]),
        "integrated_surface_change": float(integrated_change[-1]),
    }

def density_shrinkage_compatibility(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
) -> dict[str, object]:
    """Audit the hypothetical interpretation of Appendix 4 rho as wet density.

```

This is separate from the moisture PDE flux balance. With fixed length, dry mass per unit length would be  $2\pi R^2 \int \rho(C)/(1+C) \cdot \xi \, d\xi$ . The first field must be the initial field; no conservation is imposed here.

```

"""
if not np.isclose(solution.time_s[0], radius_history.time_s[0], atol=1.0e-12, rtol=0.0):
    raise ValueError("Density compatibility requires the initial-time row")
volumes = nodal_control_volumes(solution.xi.size, 1.0)
concentration = solution.moisture_concentration
implied_dry_density = density_q4(concentration) / (1.0 + concentration)
radii = interpolate_radius(solution.time_s, radius_history)
# Unit-radius annular volumes already include pi (their sum is pi).
mass_per_length = radii**2 * (implied_dry_density @ volumes)
ratio = mass_per_length / mass_per_length[0]
return {
    "interpretation": "hypothetical wet bulk density; constant cylinder length",
    "time_s": solution.time_s.tolist(),
    "implied_dry_mass_per_length_kg_m": mass_per_length.tolist(),
    "mass_ratio_to_initial": ratio.tolist(),
    "final_mass_ratio": float(ratio[-1]),
    "maximum_absolute_ratio_deviation": float(np.max(np.abs(ratio - 1.0))),
}

```